L04 — Ingestion: Sqoop và Spark JDBC từ PostgreSQL vào HDFS

Stack L04 gồm PostgreSQL (nguồn Olist, 6 bảng), HDFS, YARN, JobHistory, Sqoop 1.4.7 và Spark. Log gốc nằm trong các file `0*.txt`, ảnh ở `img/`.

File nộp

File	Nội dung
`sqoop_part1.sh`	Các lệnh Sqoop phần 1: explore, full import, TODO 1.2 (customers), append, lastmodified, saved job
`spark_jdbc_ingest.py`	TODO 1–5: đọc JDBC song song, ghi Parquet phân vùng, incremental theo watermark, dedup snapshot
`01-09_*.txt`	Log từng bước, kể cả log query phía PostgreSQL

src/spark_jdbc_ingest.py (đã hoàn thành TODO 1–5)

```
"""L04 Part 2 -- Spark JDBC ingestion from Postgres to Parquet on HDFS (STARTER -- complete the TODOs).

Run inside the spark container:
    docker compose exec spark spark-submit --jars /jars/postgresql.jar \
        /lab/src/spark_jdbc_ingest.py --step full
    docker compose exec spark spark-submit --jars /jars/postgresql.jar \
        /lab/src/spark_jdbc_ingest.py --step incremental

--step full         single-partition read vs. parallel read (partitionColumn/lowerBound/upperBound/
                    numPartitions), rows per partition, Parquet partitioned by purchase month.
--step incremental  watermark on updated_at: read only rows changed since the last run, append them
                    to a change log, rebuild the "current" snapshot, save the new watermark.
"""
import argparse
import time

from pyspark.sql import SparkSession, Window
from pyspark.sql import functions as F

JDBC_URL = "jdbc:postgresql://postgres:5432/olist"
JDBC_OPTS = {"url": JDBC_URL, "user": "olist", "password": "olist", "driver": "org.postgresql.Driver"}
BASE = "hdfs://namenode:8020/data/olist/spark"


def jdbc(spark, **options):
    """spark.read.format('jdbc') with the connection options pre-filled."""
    reader = spark.read.format("jdbc").options(**JDBC_OPTS)
    for k, v in options.items():
        reader = reader.option(k, v)
    return reader.load()


def timed(label, fn):
    t0 = time.time()
    result = fn()
    print(f"[time] {label}: {time.time() - t0:.2f} s")
    return result


def step_full(spark):
    # (1) naive read: ONE partition -> one JDBC connection, one task does all the work
    naive = jdbc(spark, dbtable="orders")
    print("naive partitions:", naive.rdd.getNumPartitions())
    timed("naive count", naive.count)

    # (2) the bounds, like Sqoop's BoundingValsQuery
    b = jdbc(spark, query="SELECT min(order_seq) AS lo, max(order_seq) AS hi FROM orders").first()
    print(f"order_seq bounds: {b.lo} .. {b.hi}")

    # (3) parallel read: Spark generates numPartitions range queries on partitionColumn.
    #     lowerBound/upperBound only decide the STRIDE -- they do NOT filter rows.
    # TODO 1: read "orders" in 4 parallel partitions split on order_seq.
    #         Use the options partitionColumn, lowerBound, upperBound (from b.lo / b.hi, as strings),
    #         numPartitions and fetchsize="1000".
    orders = jdbc(spark, dbtable="orders", partitionColumn="order_seq",
                  lowerBound=str(b.lo), upperBound=str(b.hi), numPartitions="4", fetchsize="1000")
    print("parallel partitions:", orders.rdd.getNumPartitions())
    timed("parallel count", orders.count)
    orders.groupBy(F.spark_partition_id().alias("partition")).count().orderBy("partition").show()

    # (4) same with a timestamp partition column (also allowed: numeric, date, timestamp)
    # TODO 2: same read, but split on the timestamp column order_purchase_timestamp,
    #         lowerBound "2017-01-01 00:00:00", upperBound "2018-09-01 00:00:00", 4 partitions.
    by_ts = jdbc(spark, dbtable="orders", partitionColumn="order_purchase_timestamp",
                 lowerBound="2017-01-01 00:00:00", upperBound="2018-09-01 00:00:00", numPartitions="4")
    print("rows per partition when splitting on order_purchase_timestamp (volume grows over time):")
    by_ts.groupBy(F.spark_partition_id().alias("partition")).count().orderBy("partition").show()

    # (5) write Parquet, partitioned by purchase month (Hive-style directories)
    # TODO 3: add a column purchase_month = 'yyyy-MM' of order_purchase_timestamp and write the
    #         DataFrame as Parquet to f"{BASE}/orders", partitioned by purchase_month, mode overwrite.
    out = orders.withColumn("purchase_month", F.date_format("order_purchase_timestamp", "yyyy-MM"))
    timed("write parquet", lambda: out.write.mode("overwrite").partitionBy("purchase_month")
          .parquet(f"{BASE}/orders"))
    back = spark.read.parquet(f"{BASE}/orders")
    print("rows written:", back.count(), "| months:", back.select("purchase_month").distinct().count())
    back.filter("purchase_month = '2018-08'").explain()   # look for PartitionFilters

    # (6) the other tables, one call each (small tables: 1 partition is fine)
    for table in ["customers", "order_items", "order_payments", "products", "sellers"]:
        df = jdbc(spark, dbtable=table)
        df.write.mode("overwrite").parquet(f"{BASE}/{table}")
        print(f"{table}: {df.count()} rows")


def path_exists(spark, path):
    """Ask the Hadoop FileSystem API (through py4j) whether an HDFS path exists."""
    jpath = spark._jvm.org.apache.hadoop.fs.Path(path)
    return jpath.getFileSystem(spark._jsc.hadoopConfiguration()).exists(jpath)


def read_watermark(spark):
    state = f"{BASE}/_state/orders_watermark"
    if not path_exists(spark, state):          # first run: no state yet -> load everything
        return "1970-01-01 00:00:00"
    return spark.read.json(state).first()["watermark"]


def step_incremental(spark):
    wm_old = read_watermark(spark)
    # upper bound = DB clock NOW, fixed before reading, so the next run starts exactly here
    wm_new = jdbc(spark, query="SELECT to_char(now(), 'YYYY-MM-DD HH24:MI:SS.US') AS ts").first().ts
    print(f"watermark: ({wm_old}, {wm_new}]")

    # TODO 4: read ONLY the rows with  wm_old < updated_at <= wm_new.
    #         Hint: dbtable accepts a sub-query with an alias: "(SELECT ... WHERE ...) AS changed_orders"
    #         Then add a column _ingested_at = wm_new cast to timestamp.
    changed_sql = (f"(SELECT * FROM orders WHERE updated_at > timestamp '{wm_old}'"
                   f" AND updated_at <= timestamp '{wm_new}') AS changed_orders")
    changes = jdbc(spark, dbtable=changed_sql).withColumn("_ingested_at", F.lit(wm_new).cast("timestamp"))
    changes = changes.cache()
    n = changes.count()
    print(f"changed rows since last run: {n}")
    changes.select("order_seq", "order_id", "order_status", "updated_at").orderBy("updated_at").show(10, False)

    # append-only change log (bronze-style), one directory per ingestion
    changes.write.mode("append").parquet(f"{BASE}/orders_changes")

    # current snapshot = latest version of every order_id across the whole change log
    log = spark.read.parquet(f"{BASE}/orders_changes")
    # TODO 5: keep only the LATEST version of every order_id (row_number over a window partitioned by
    #         order_id, ordered by updated_at desc, _ingested_at desc; keep row 1).
    w = Window.partitionBy("order_id").orderBy(F.col("updated_at").desc(), F.col("_ingested_at").desc())
    current = log.withColumn("_rn", F.row_number().over(w)).filter("_rn = 1").drop("_rn")
    current.write.mode("overwrite").parquet(f"{BASE}/orders_current")
    print("change-log rows:", log.count(), "| current snapshot rows:",
  spark.read.parquet(f"{BASE}/orders_current").count())

    # persist the new watermark ONLY after the data was written successfully
    spark.createDataFrame([(wm_new,)], "watermark string").coalesce(1) \
        .write.mode("overwrite").json(f"{BASE}/_state/orders_watermark")
    print("saved watermark", wm_new)

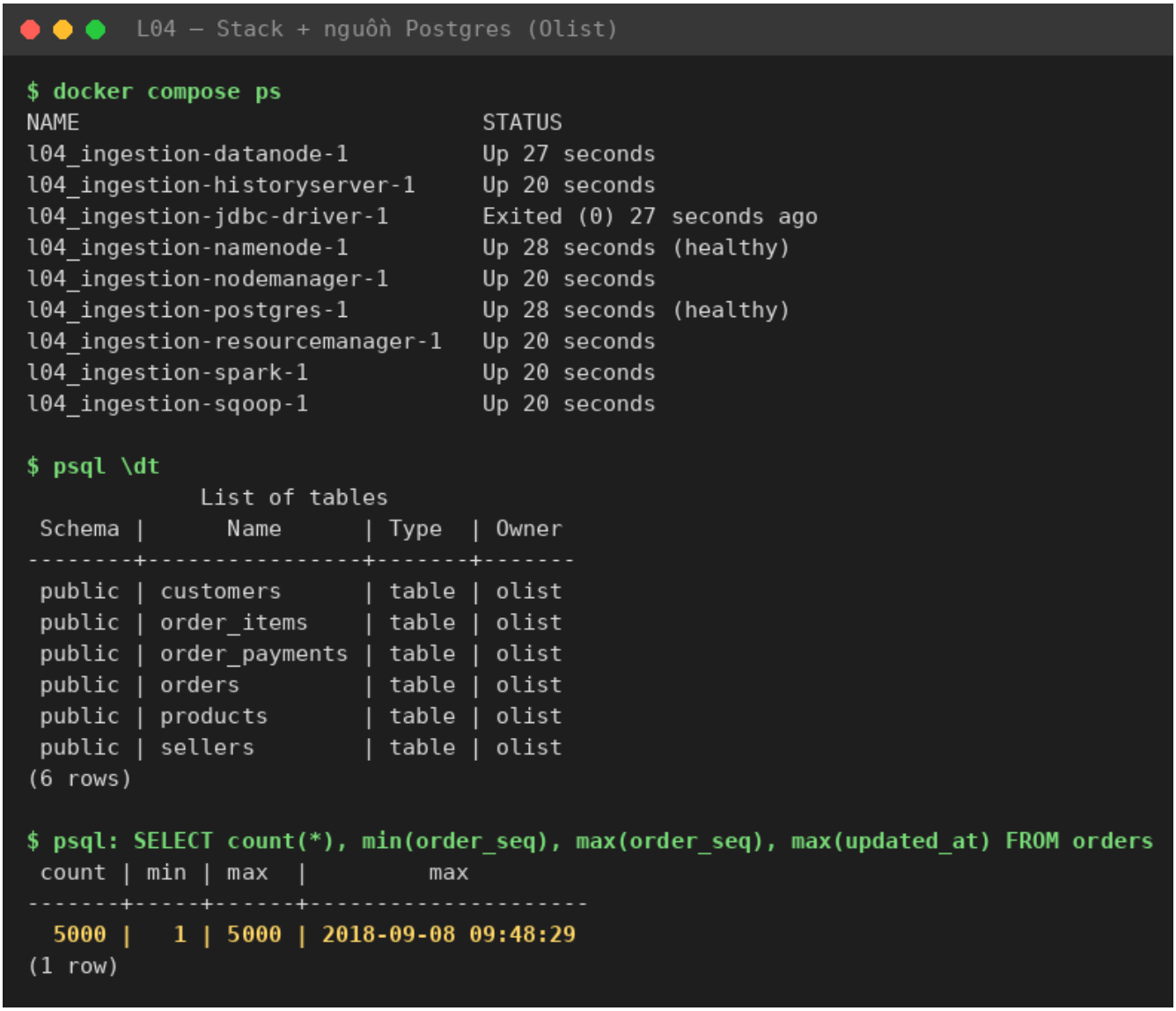

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--step", choices=["full", "incremental"], required=True)
    args = ap.parse_args()
    spark = (SparkSession.builder.appName(f"L04-jdbc-{args.step}")
             .config("spark.hadoop.dfs.replication", "1")      # single-DataNode lab HDFS
             .config("spark.sql.session.timeZone", "UTC")
             .getOrCreate())
    spark.sparkContext.setLogLevel("WARN")
    {"full": step_full, "incremental": step_incremental}[args.step](spark)
    spark.stop()


if __name__ == "__main__":
    main()
```

Hình 1: spark_jdbc_ingest.py

1. Nguồn dữ liệu

Có 6 bảng. `orders` có 5,000 dòng, `order_seq` chạy từ 1 đến 5000 và `max(updated_at)` là `2018-09-08 09:48:29`. Theo trạng thái: delivered 4,793, shipped 102, ...



Hình 2: stack and source

2. Sqoop full import và cách chia split

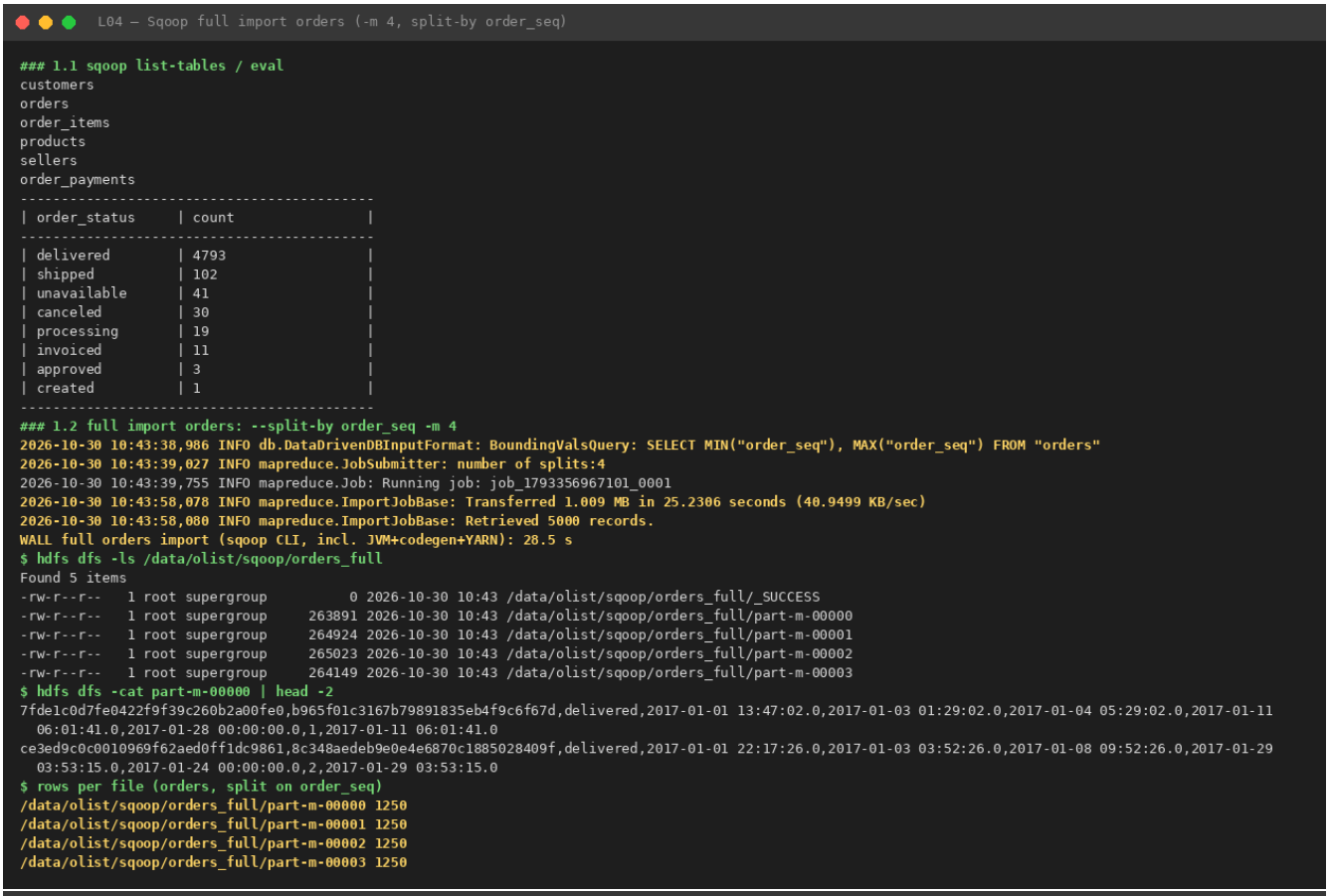
`sqoop import --table orders --split-by order_seq -m 4`:

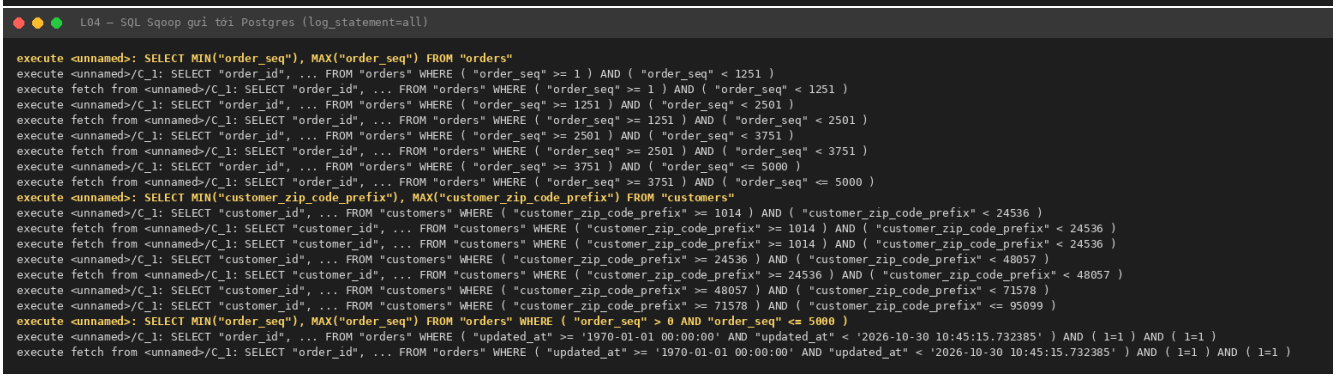
- Sqoop chạy `BoundingValsQuery` (`SELECT MIN(order_seq), MAX(order_seq) FROM orders`), được (1, 5000), rồi chia thành 4 khoảng đều nhau.
- Log PostgreSQL ghi lại đúng 4 câu query:

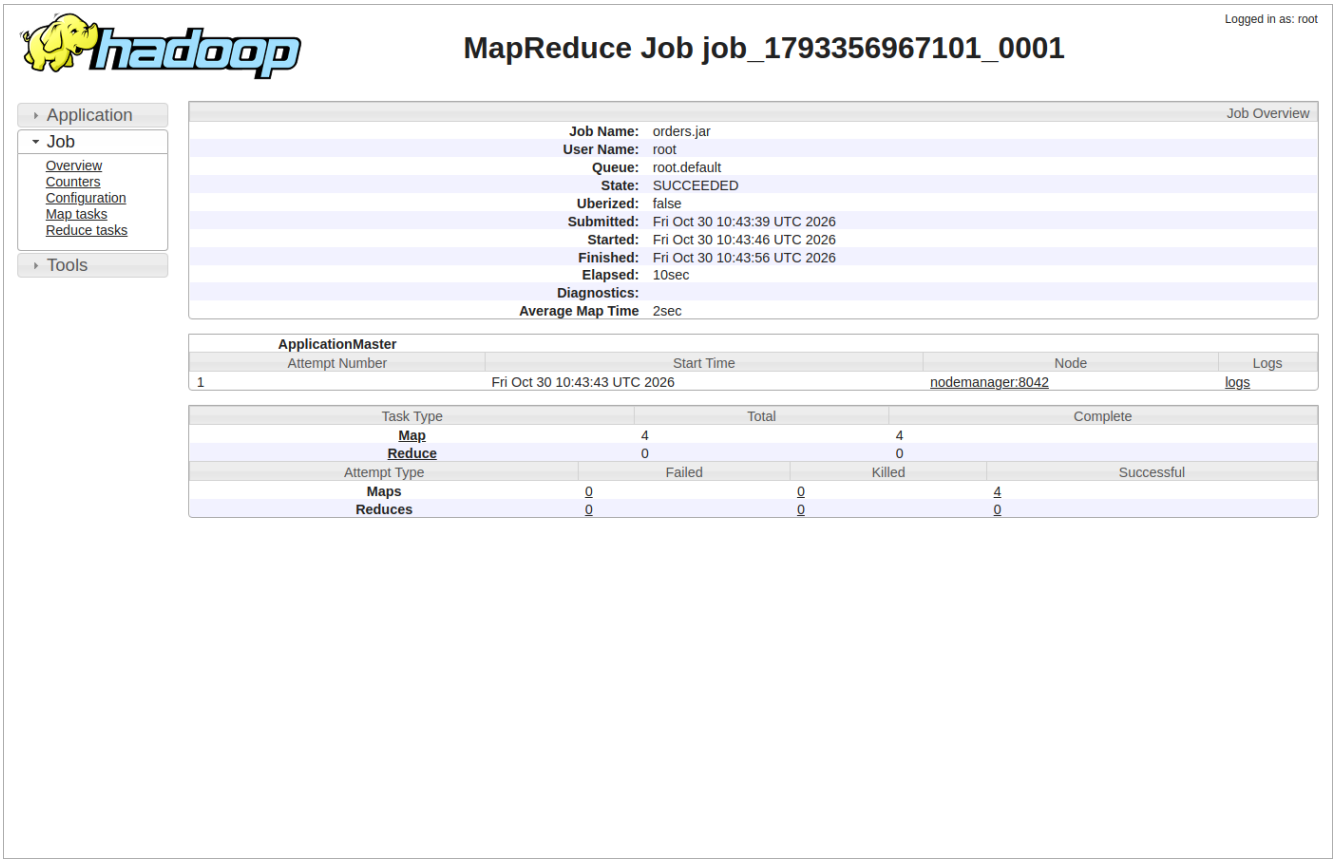
Map	Điều kiện WHERE	Số dòng
0	`order_seq >= 1 AND order_seq < 1251`	1,250
1	`order_seq >= 1251 AND order_seq < 2501`	1,250

Map	Điều kiện WHERE	Số dòng
2	`order_seq >= 2501 AND order_seq < 3751`	1,250
3	`order_seq >= 3751 AND order_seq <= 5000`	1,250

- Đã chuyển 1.009 MB trong 25.23 s (thời gian MR); tổng thời gian lệnh CLI là 28.5 s.
- 4 file `part-m-0000x` chia đều nhau.



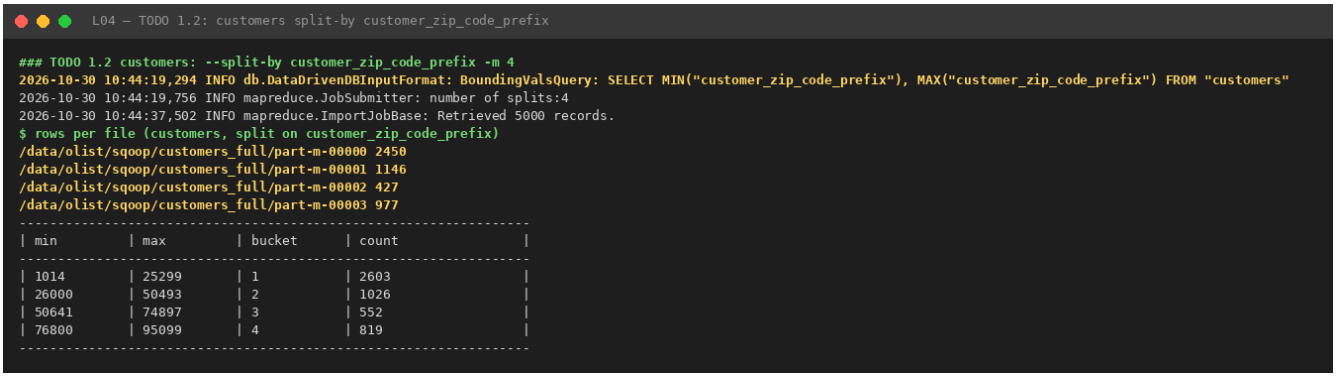




TODO 1.2: `customers --split-by customer_zip_code_prefix -m 4`

Bảng / cột split	part-m-00000	part-m-00001	part-m-00002	part-m-00003
`orders` / `order_seq` (1–5000)	1,250	1,250	1,250	1,250
`customers` / `customer_zip_code_prefix` (1014–95099)	**2,450**	1,146	**427**	977

Khoảng zip được chia đều tại 24536, 48057 và 71578, nhưng zip code tập trung ở vùng São Paulo (01xxx–19xxx). Vì vậy split đầu nhận gần một nửa số dòng, còn split thứ ba chỉ nhận 427 dòng: **data skew**.



Hình 3: customers skew

3. Sqoop incremental: append, lastmodified, saved job, merge, export

Bước	Kết quả
Append, lần nạp đầu	lower 0, upper 5000 → 5,000 dòng; Sqoop gợi ý `--last-value 5000`
Lastmodified, baseline	5,000 dòng; last-value `2026-10-30 10:45:15.732385`
Saved job `orders_incr`	`incremental.last.value = 5000` lưu trong metastore của Sqoop
`simulate_changes` #1	tạo `order_seq` 5001–5003; cập nhật đơn 139 và 278 thành `delivered`
`sqoop job --exec orders_incr`	lower 5000, upper 5003 → lấy **3** dòng; `last.value` thành 5003; tổng 5,003 dòng
Lastmodified + `--merge-key order_id`	lấy **5** dòng (3 mới + 2 cập nhật); merge job → một `part-r-00000` gồm 5,003 dòng, không trùng
`sqoop export` ngược về PostgreSQL	5,000 record

Saved job tự lưu `last.value`, nên lần chạy sau chỉ đọc phần mới. Append chỉ nhìn thấy 3 dòng INSERT. Lastmodified thấy thêm 2 dòng UPDATE, và merge-key gộp mỗi `order_id` về bản mới nhất.

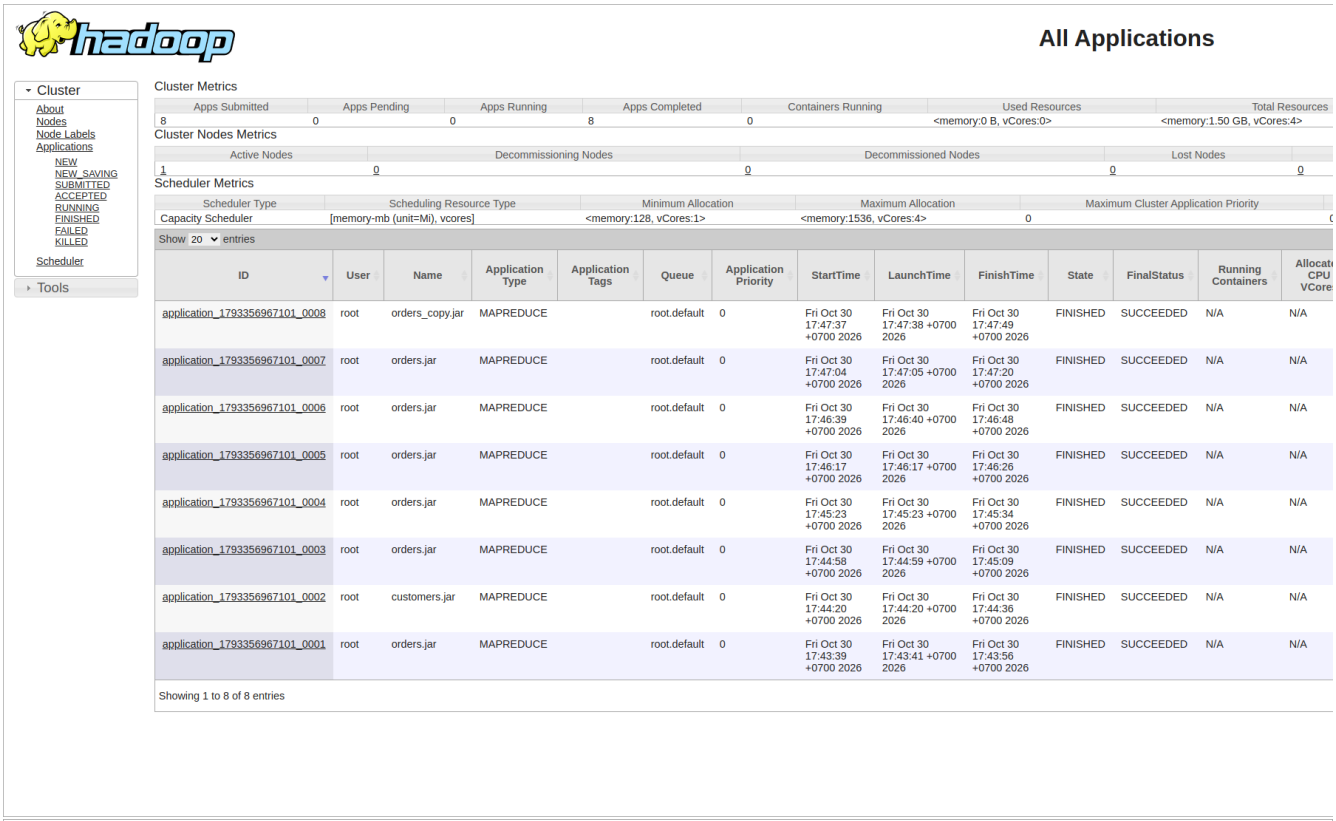
L04 — Sqoop incremental append / lastmodified baseline / saved job

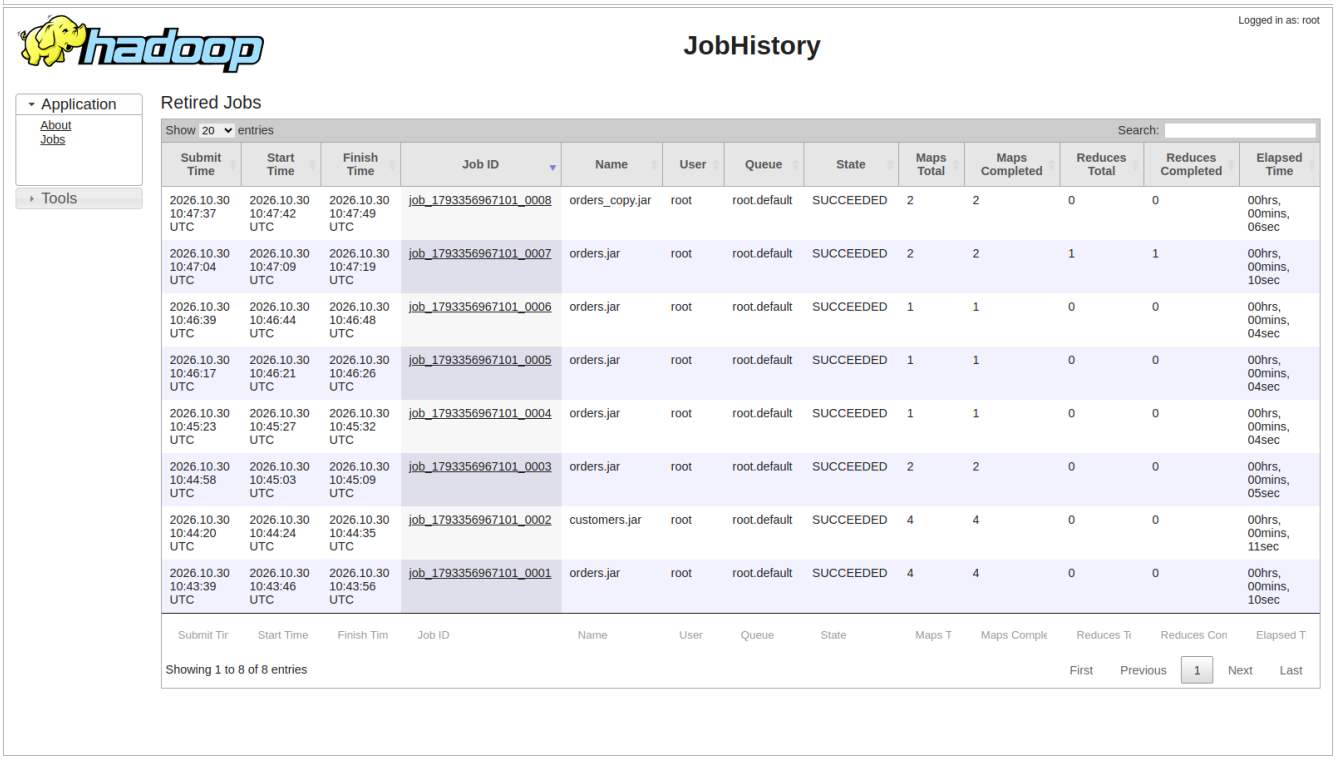
```
### 1.3 incremental append (initial load, --last-value 0)
2026-10-30 10:44:51,951 INFO tool.ImportTool: Lower bound value: 0
2026-10-30 10:44:51,951 INFO tool.ImportTool: Upper bound value: 5000
2026-10-30 10:45:10,960 INFO mapreduce.ImportJobBase: Retrieved 5000 records.
2026-10-30 10:45:10,989 INFO tool.ImportTool:   --incremental append
2026-10-30 10:45:10,989 INFO tool.ImportTool:    --check-column order_seq
2026-10-30 10:45:10,989 INFO tool.ImportTool:    --last-value 5000
2026-10-30 10:45:10,989 INFO tool.ImportTool: (Consider saving this with 'sqoop job --create')
### 1.5a incremental lastmodified baseline
2026-10-30 10:45:15,734 INFO tool.ImportTool: Lower bound value: '1970-01-01 00:00:00'
2026-10-30 10:45:15,734 INFO tool.ImportTool: Upper bound value: '2026-10-30 10:45:15.732385'
2026-10-30 10:45:35,734 INFO mapreduce.ImportJobBase: Retrieved 5000 records.
2026-10-30 10:45:35,746 INFO tool.ImportTool:    --last-value 2026-10-30 10:45:15.732385
next --last-value: 2026-10-30 10:45:15.732385
### 1.4 saved job orders_incr
$ sqoop job --list
Available jobs:
  orders_incr
$ sqoop job --show orders_incr | grep incremental
incremental.last.value = 5000
incremental.col = order_seq
incremental.mode = AppendRows
```

L04 — simulate_changes → saved job exec, lastmodified+merge, export

```
$ psql -f sql/simulate_changes.sql   (#1, before Sqoop part 2)
INSERT 0 3
UPDATE 2
 order_seq |             order_id             | order_status |         updated_at
-----------+----------------------------------+--------------+----------------------------
      5001 | b66d7d8bbcad4568830448d9d663fac5 | created      | 2026-10-30 10:46:07.357187
      5002 | 0c83fcfb71e8bc5b9e8b3a73f7140d4f | created      | 2026-10-30 10:46:07.357187
      5003 | 7008b2a0e1ffae9397506c90fb4eab1b | created      | 2026-10-30 10:46:07.357187
       139 | 8a7ac9a44f356814c2311987dcd8ad10 | delivered    | 2026-10-30 10:46:07.361794
       278 | 65a11c7df0f6c77c1688232a1107044f | delivered    | 2026-10-30 10:46:07.361794
(5 rows)


### 4b. exec the saved job -> only the NEW orders are appended
2026-10-30 10:46:10,820 INFO tool.ImportTool: Lower bound value: 5000
2026-10-30 10:46:10,821 INFO tool.ImportTool: Upper bound value: 5003
2026-10-30 10:46:28,335 INFO mapreduce.ImportJobBase: Retrieved 3 records.
incremental.last.value = 5003
Found 3 items
-rw-r--r--   1 root supergroup     528815 2026-10-30 10:45 /data/olist/sqoop/orders_append/part-m-00000
-rw-r--r--   1 root supergroup     529172 2026-10-30 10:45 /data/olist/sqoop/orders_append/part-m-00001
-rw-r--r--   1 root supergroup        420 2026-10-30 10:46 /data/olist/sqoop/orders_append/part-m-00002
rows in append dir: 5003
### 5. incremental LASTMODIFIED + --merge-key: new AND updated rows, merged into the snapshot
using --last-value '2026-10-30 10:45:15.732385'
2026-10-30 10:46:35,899 INFO tool.ImportTool: Lower bound value: '2026-10-30 10:45:15.732385'
2026-10-30 10:46:35,899 INFO tool.ImportTool: Upper bound value: '2026-10-30 10:46:35.897666'
                Merged Map outputs=0
2026-10-30 10:46:50,942 INFO mapreduce.ImportJobBase: Retrieved 5 records.
2026-10-30 10:46:50,945 INFO tool.ImportTool: Final destination exists, will run merge job.
                Merged Map outputs=2
2026-10-30 10:47:21,419 INFO tool.ImportTool:   --last-value 2026-10-30 10:46:35.897666
Found 2 items
-rw-r--r--   1 root supergroup          0 2026-10-30 10:47 /data/olist/sqoop/orders_lastmod/_SUCCESS
-rw-r--r--   1 root supergroup    1058459 2026-10-30 10:47 /data/olist/sqoop/orders_lastmod/part-r-00000
rows in lastmod dir: 5003   (= baseline + the new orders; updated orders were replaced in place, not duplicated)
0c83fcfb71e8bc5b9e8b3a73f7140d4f,201747a216850606f49c716719a837ad,created,2026-10-30 10:46:07.0,\N,\N,\N,\N,5002,2026-10-30 10:46:07.357187
65a11c7df0f6c77c1688232a1107044f,d51eb5503b9ff6e63bb39685d32577a2,delivered,2017-03-03 11:33:42.0,2017-03-03 11:56:42.0,2017-03-04 15:56:42.0,2026-10-30 10:46:07.0,2017-03-23
  00:00:00.0,278,2026-10-30 10:46:07.361794
7008b2a0e1ffae9397506c90fb4eab1b,af3b0010d669e4b210716eae69c81c2d,created,2026-10-30 10:46:07.0,\N,\N,\N,\N,5003,2026-10-30 10:46:07.357187
8a7ac9a44f356814c2311987dcd8ad10,155e17e9b120e25f6a64f2c8efe2a650,delivered,2017-02-03 10:49:42.0,2017-02-03 10:54:42.0,2017-02-09 09:54:42.0,2026-10-30 10:46:07.0,2017-02-23
  00:00:00.0,139,2026-10-30 10:46:07.361794
b66d7d8bbcad4568830448d9d663fac5,a26322435a248248b86845ebe0587466,created,2026-10-30 10:46:07.0,\N,\N,\N,\N,5001,2026-10-30 10:46:07.357187
### 6. export: HDFS -> new Postgres table orders_copy
2026-10-30 10:47:51,093 INFO mapreduce.ExportJobBase: Transferred 1.0094 MB in 18.4353 seconds (56.0684 KB/sec)
2026-10-30 10:47:51,095 INFO mapreduce.ExportJobBase: Exported 5000 records.
-----------------------
| rows_exported       |
-----------------------
| 5000                |
-----------------------
```





4. Spark JDBC: đọc song song và ghi Parquet (TODO 1–3)

Cách đọc	Số partition	Thời gian `count()`	Phân bố dòng
Naive (không có `partitionColumn`)	1	1.17 s	tất cả 5,003 dòng trong 1 task
`partitionColumn=order_seq`, bounds 1..5003, 4 partitions	4	0.22 s	1,251 / 1,250 / 1,250 / 1,252
`partitionColumn=order_purchase_timestamp` (2017-01-01 → 2018-09-01)	4	—	782 / 1,116 / 1,419 / 1,686

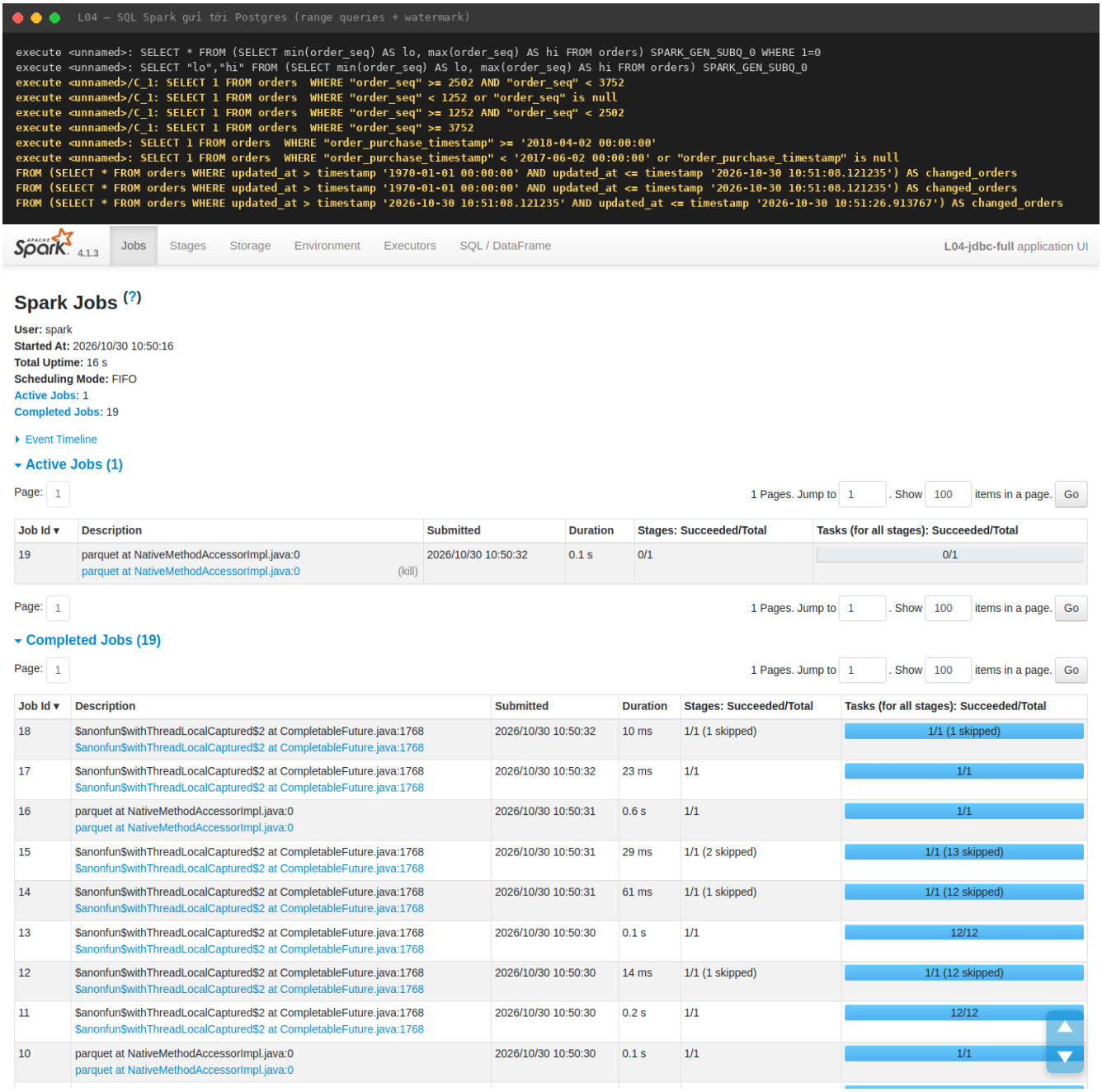
- Spark tự sinh 4 câu SQL (log PostgreSQL): `WHERE "order_seq" < 1252 or "order_seq" is null`, … , `WHERE "order_seq" >= 3752`. Partition đầu và cuối để **hở**, nên mọi dòng ngoài bounds vẫn được đọc.
- Split theo timestamp bị lệch vì số đơn tăng dần theo thời gian.
- Ghi Parquet `partitionBy("purchase_month")`:
  - mất 3.10 s, ra 21 thư mục tháng, đủ 5,003 dòng;
  - query lọc `purchase_month = '2018-08'` cho thấy `PartitionFilters` trong physical plan, tức là có partition pruning.
- Các bảng khác:

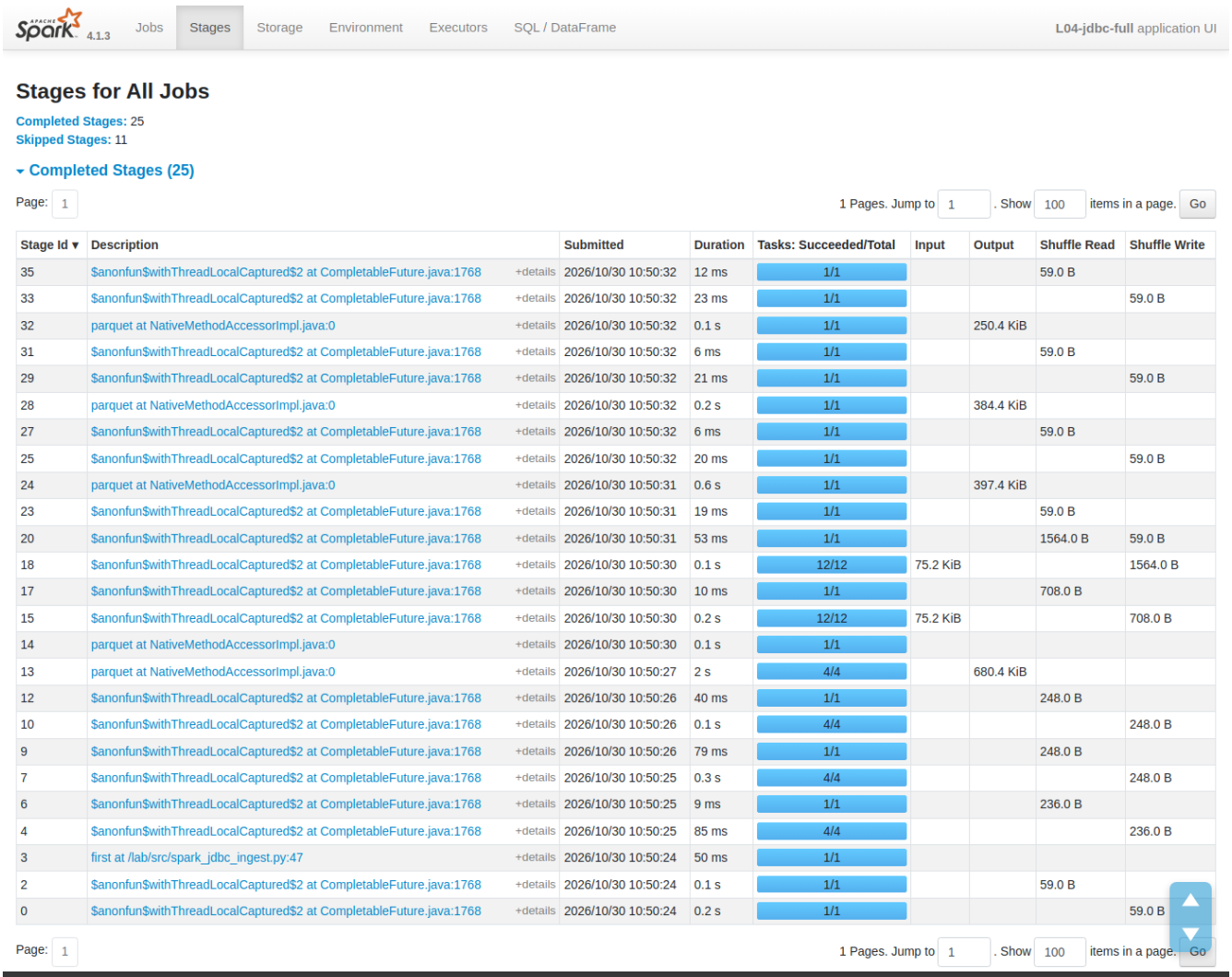
Bảng	Số dòng
customers	5,003
order_items	5,852
order_payments	5,472
products	1,500
sellers	312

- Toàn bộ `spark-submit ... full` chạy hết 21.2 s.

```
L04 — spark-submit spark_jdbc_ingest.py --step full

naive partitions: 1
[time] naive count: 1.17 s
order_seq bounds: 1 .. 5003
parallel partitions: 4
[time] parallel count: 0.22 s
+---------+-----+
|partition|count|
+---------+-----+
|        0| 1251|
|        1| 1250|
|        2| 1250|
|        3| 1252|
+---------+-----+
rows per partition when splitting on order_purchase_timestamp (volume grows over time):
+---------+-----+
|partition|count|
+---------+-----+
|        0|  782|
|        1| 1116|
|        2| 1419|
|        3| 1686|
+---------+-----+
[time] write parquet: 3.10 s
rows written: 5003 | months: 21
== Physical Plan ==
*(1) ColumnarToRow
+- FileScan parquet [order_id#108,customer_id#109,order_status#110,order_purchase_timestamp#111,order_approved_at#112,order_delivered_carrier_date#113,order_delivered_customer_date
  #114,order_estimated_delivery_date#115,order_seq#116L,updated_at#117,purchase_mo
customers: 5003 rows
order_items: 5852 rows
order_payments: 5472 rows
products: 1500 rows
sellers: 312 rows
WALL spark-submit --step full (incl. JVM start): 21.2 s
```


L04 – SQL Spark gửi tới Postgres (range queries + watermark)
execute <unnamed>: SELECT * FROM (SELECT min(order_seq) AS lo, max(order_seq) AS hi FROM orders) SPARK_GEN_SUBQ_0 WHERE 1=0
execute <unnamed>: SELECT "lo","hi" FROM (SELECT min(order_seq) AS lo, max(order_seq) AS hi FROM orders) SPARK_GEN_SUBQ_0
execute <unnamed>/C_1: SELECT 1 FROM orders WHERE "order_seq" >= 2502 AND "order_seq" < 3752
execute <unnamed>/C_1: SELECT 1 FROM orders WHERE "order_seq" < 1252 or "order_seq" is null
execute <unnamed>/C_1: SELECT 1 FROM orders WHERE "order_seq" >= 1252 AND "order_seq" < 2502
execute <unnamed>/C_1: SELECT 1 FROM orders WHERE "order_seq" >= 3752
execute <unnamed>: SELECT 1 FROM orders WHERE "order_purchase_timestamp" >= '2018-04-02 00:00:00'
execute <unnamed>: SELECT 1 FROM orders WHERE "order_purchase_timestamp" < '2017-06-02 00:00:00' or "order_purchase_timestamp" is null
FROM (SELECT * FROM orders WHERE updated_at > timestamp '1970-01-01 00:00:00' AND updated_at <= timestamp '2026-10-30 10:51:08.121235') AS changed_orders
FROM (SELECT * FROM orders WHERE updated_at > timestamp '1970-01-01 00:00:00' AND updated_at <= timestamp '2026-10-30 10:51:08.121235') AS changed_orders
FROM (SELECT * FROM orders WHERE updated_at > timestamp '2026-10-30 10:51:08.121235' AND updated_at <= timestamp '2026-10-30 10:51:26.913767') AS changed_orders
Spark 4.1.3
Jobs
Stages
Storage
Environment
Executors
SQL / DataFrame
L04-jdbc-full application UI
Spark Jobs (?)
User: spark
Started At: 2026/10/30 10:50:16
Total Uptime: 16 s
Scheduling Mode: FIFO
Active Jobs: 1
Completed Jobs: 19
Event Timeline
Active Jobs (1)
Page: 1
1 Pages. Jump to 1 . Show 100 items in a page. Go
Job Id ▾
Description
Submitted
Duration
Stages: Succeeded/Total
Tasks (for all stages): Succeeded/Total
19
parquet at NativeMethodAccessorImpl.java:0
parquet at NativeMethodAccessorImpl.java:0 (kill)
2026/10/30 10:50:32
0.1 s
0/1
0/1
Page: 1
1 Pages. Jump to 1 . Show 100 items in a page. Go
Completed Jobs (19)
Page: 1
1 Pages. Jump to 1 . Show 100 items in a page. Go
Job Id ▾
Description
Submitted
Duration
Stages: Succeeded/Total
Tasks (for all stages): Succeeded/Total
18
$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768
$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768
2026/10/30 10:50:32
10 ms
1/1 (1 skipped)
1/1 (1 skipped)
17
$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768
$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768
2026/10/30 10:50:32
23 ms
1/1
1/1
16
parquet at NativeMethodAccessorImpl.java:0
parquet at NativeMethodAccessorImpl.java:0
2026/10/30 10:50:31
0.6 s
1/1
1/1
15
$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768
$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768
2026/10/30 10:50:31
29 ms
1/1 (2 skipped)
1/1 (13 skipped)
14
$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768
$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768
2026/10/30 10:50:31
61 ms
1/1 (1 skipped)
1/1 (12 skipped)
13
$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768
$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768
2026/10/30 10:50:30
0.1 s
1/1
12/12
12
$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768
$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768
2026/10/30 10:50:30
14 ms
1/1 (1 skipped)
1/1 (12 skipped)
11
$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768
$anonfun$withThreadLocalCaptured$2 at CompletableFuture.java:1768
2026/10/30 10:50:30
0.2 s
1/1
12/12
10
parquet at NativeMethodAccessorImpl.java:0
parquet at NativeMethodAccessorImpl.java:0
2026/10/30 10:50:30
0.1 s
1/1
1/1




```
L04 — Parquet partitionBy(purchase_month) trên HDFS

$ hdfs dfs -ls /data/olist/spark/orders | head
Found 22 items
-rw-r--r--   1 spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/_SUCCESS
drwxr-xr-x   - spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/purchase_month=2017-01
drwxr-xr-x   - spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/purchase_month=2017-02
drwxr-xr-x   - spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/purchase_month=2017-03
drwxr-xr-x   - spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/purchase_month=2017-04
drwxr-xr-x   - spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/purchase_month=2017-05
drwxr-xr-x   - spark supergroup          0 2026-10-30 10:50 /data/olist/spark/orders/purchase_month=2017-06
...
$ hdfs dfs -du -s -h /data/olist/spark/orders /data/olist/sqoop/orders_full
680.4 K  680.4 K  /data/olist/spark/orders
1.0 M    1.0 M    /data/olist/sqoop/orders_full
```

So sánh Sqoop và Spark JDBC (orders)

	Sqoop (MapReduce)	Spark JDBC
Song song	`-m 4`, chia theo MIN/MAX của `--split-by`	`numPartitions=4`, chia theo `lowerBound`/`upperBound` do mình truyền vào

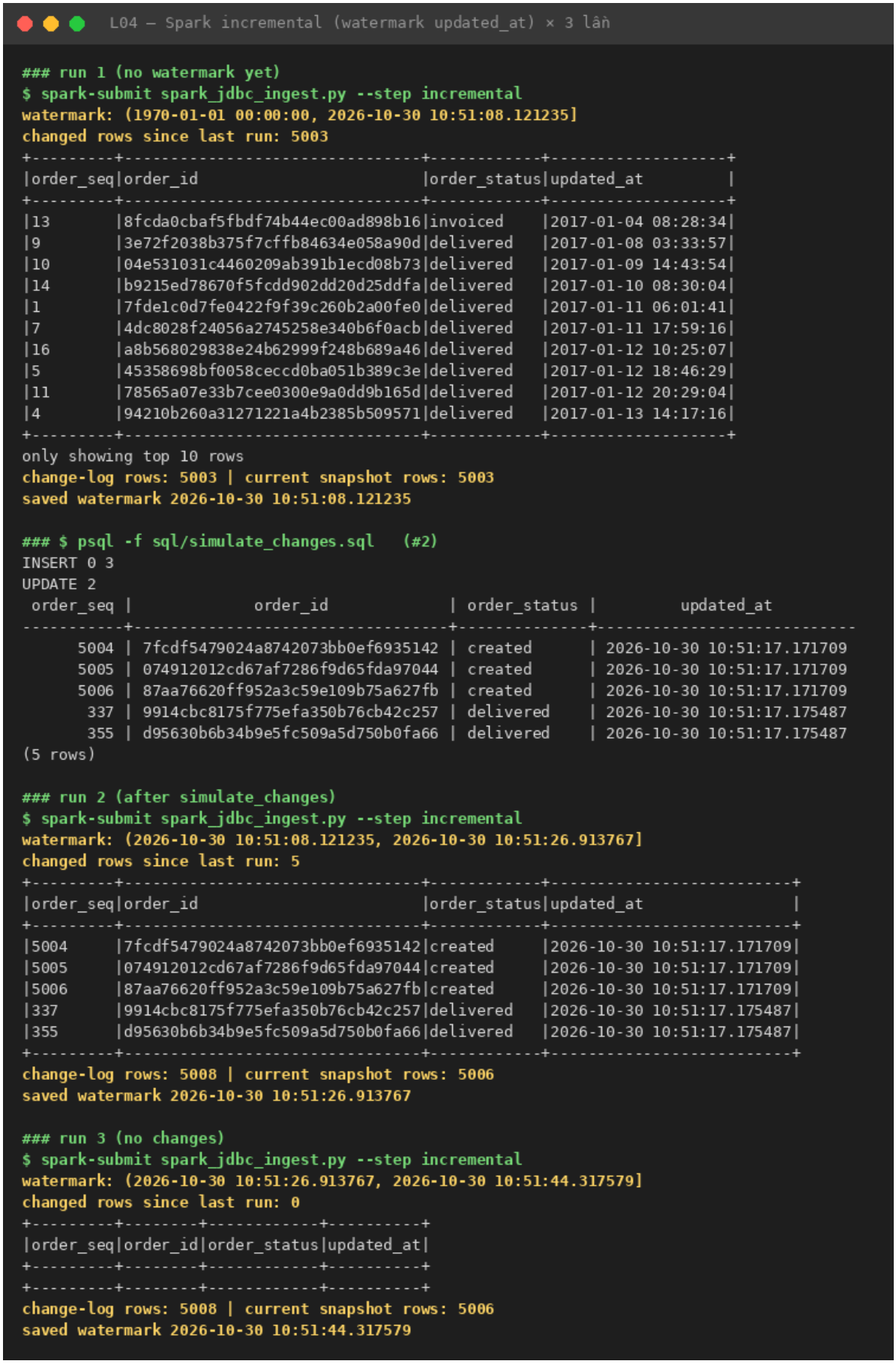
	Sqoop (MapReduce)	Spark JDBC
Thời gian	25.2 s MR job (28.5 s cả lệnh)	3.10 s ghi Parquet (21.2 s cả `spark-submit` 7 bảng, tính cả khởi động JVM/YARN)
Định dạng ra	Text CSV, 1.0 MB	Parquet Snappy phân vùng theo tháng, 680.4 KB
Incremental	Có sẵn (`--incremental`, saved job, merge)	Phải tự viết: watermark, change log, dedup
Biến đổi trong lúc nạp	Không	Có (`withColumn`, `partitionBy`, …)

Sqoop chậm chủ yếu vì phải khởi động một MR job riêng cho mỗi bảng. Spark dùng chung một SparkSession cho cả 7 bảng và ghi thẳng ra định dạng cột. Sqoop 1 cũng đã ngừng phát triển (Apache attic), nên ngày nay thường dùng Spark JDBC hoặc CDC.

5. Spark incremental: watermark, change log, snapshot (TODO 4–5)

Lần chạy	Cửa sổ watermark	Dòng đọc được	Change log	Snapshot
1 (lần đầu)	`(1970-01-01, 2026-10-30 10:51:08.121235]`	5,003	5,003	5,003
sau `simulate_changes` #2 (tạo 5004–5006, cập nhật 337 và 355)				
2	`(10:51:08.121235, now]`	**5**	5,008	**5,006**
3 (không có thay đổi)	`(…, now]`	**0**	5,008	5,006

- Change log chỉ append và giữ mọi phiên bản: 5,003 + 5 = 5,008.
- Snapshot dùng `row_number()` theo `order_id`, sắp `updated_at DESC, _ingested_at DESC`, nên mỗi đơn chỉ còn bản mới nhất: 5,006 đơn.
- Lần chạy thứ ba đọc 0 dòng, nghĩa là watermark được lưu đúng và pipeline idempotent.



Hình 4: spark incremental

Trả lời checkpoint

Q1. Trước khi import, Sqoop chạy bounding query `SELECT MIN(order_seq), MAX(order_seq) FROM orders` (thấy rõ trong log PostgreSQL), được (1, 5000). Nó chia khoảng này thành `-m 4` đoạn **bằng nhau về giá trị**, mỗi đoạn rộng (5000 − 1)/4 ≈ 1250, và mỗi map task chạy một câu `SELECT … WHERE order_seq >= lo AND order_seq < hi`. Đoạn cuối dùng `<=` để lấy cả MAX. Vì `order_seq` dày đặc từ 1 đến 5000, mỗi map nhận đúng 1,250 dòng.

Q2. Sqoop chia theo **khoảng giá trị**, không theo số dòng. Zip code của khách Olist tập trung ở São Paulo (01xxx–19xxx), nên đoạn [1014, 24536) nhận 2,450 dòng còn đoạn [48057, 71578) chỉ có 427 dòng. Task chậm nhất quyết định thời gian của cả job. `customer_id` là chuỗi hash dạng text. Sqoop mặc định từ chối split theo cột text và báo lỗi, trừ khi bật `-Dorg.apache.sqoop.splitter.allow_text_splitter=true`. Kể cả khi bật, việc chia theo thứ tự chuỗi vẫn kém tin cậy (phụ thuộc collation), và split có thể lệch hoặc rỗng. Nên chọn một cột số, phân bố đều và có index, như `order_seq`.

Q3. Có. `lowerBound`/`upperBound` chỉ dùng để tính **bước chia** (stride), **không phải bộ lọc**. Spark sinh partition đầu là `order_seq < 1252 OR order_seq IS NULL` và partition cuối là `order_seq >= 3752`, không có cận trên. Vì vậy dòng `order_seq = 9,999` vẫn được đọc và rơi vào **partition cuối**. Nếu nhiều dòng nằm ngoài bounds, partition đầu hoặc cuối sẽ phình to và gây skew. Phải lấy bounds thật (MIN/MAX) trước khi đọc, như script đã làm (`bounds 1..5003`).

Q4. - **Chốt cận trên trước khi đọc:** `wm_new = now()` được lấy trước khi đọc, và query dùng `updated_at > wm_old AND updated_at <= wm_new`. Nếu lấy `now()` sau khi đọc, những dòng được cập nhật **trong lúc** đang đọc sẽ có `updated_at` nằm trước watermark mới mà không nằm trong dữ liệu đã đọc. Lần chạy sau sẽ bỏ qua chúng vĩnh viễn: **mất dữ liệu**. Cận trên cố định cũng giúp các partition JDBC song song nhìn cùng một cửa sổ, và lần sau bắt đầu đúng từ chỗ lần này dừng. - **Chỉ lưu watermark sau khi ghi thành công:** nếu job chết giữa chừng, watermark cũ vẫn còn và lần chạy lại sẽ đọc lại cả cửa sổ. Như vậy dữ liệu được giao **at-least-once**: có thể trùng nhưng không mất. Phần trùng được dedup idempotent bằng `row_number()` theo `order_id` khi dựng snapshot. Nếu lưu watermark trước khi ghi mà ghi thất bại, cửa sổ đó sẽ bị bỏ qua mãi mãi.

Q5. `append` chỉ dựa vào `order_seq > last_value`, nên chỉ bắt được **INSERT** (lần chạy job chỉ lấy 3 dòng mới). `lastmodified` dựa vào `updated_at > last_value`, nên bắt được cả **INSERT và UPDATE** (lấy 5 dòng: 3 mới + 2 đơn 139 và 278). Cần `--merge-key` để gộp bản cũ với bản mới. **Không chế độ nào bắt được DELETE**: dòng đã xoá thì không còn trong bảng để query ra.

Q6. Watermark trên `updated_at` **không** giải quyết được: 1. **DELETE**: dòng bị xoá không bao giờ xuất hiện trong query, nên bản sao trên HDFS giữ dữ liệu "ma" (trừ khi ứng dụng dùng soft delete). 2. **Các lần UPDATE trung gian**: nếu một dòng đổi nhiều lần giữa hai lần chạy (ví dụ `shipped` → `delivered` → `canceled`), ta chỉ thấy trạng thái cuối và mất lịch sử. 3. **`updated_at` không đáng tin**: - ứng dụng quên cập nhật cột, hoặc ghi bằng giờ của app server lệch đồng hồ; - transaction dài commit sau khi watermark đã vượt qua timestamp của nó, nên dòng bị bỏ sót. 4. **Tải lên nguồn**: mỗi lần chạy là một query quét theo `updated_at` (cần index), độ trễ bằng chu kỳ poll.

CDC (Debezium, L08) không query bảng mà đọc **transaction log (WAL)** của PostgreSQL qua logical replication. Mọi INSERT, UPDATE và DELETE được phát ra thành event theo đúng thứ tự commit, kèm ảnh trước và sau. Không phụ thuộc cột `updated_at`, gần như không tạo tải query, và độ trễ chỉ vài giây.

Q7. `-m 50` hoặc `numPartitions=50` mở **50 kết nối đồng thời**, mỗi kết nối chạy một range scan.

Như vậy dễ vượt `max_connections` (PostgreSQL mặc định 100), làm ứng dụng production không lấy được kết nối. Nó cũng tranh CPU, I/O và buffer cache với tải OLTP, và với bảng nhỏ (5,000 dòng) thì mỗi task chỉ có ~100 dòng, chi phí khởi tạo lấn át lợi ích. Thực tế nên giới hạn khoảng 4–8 kết nối, đọc từ **read replica** hoặc ngoài giờ cao điểm, và đặt `fetchsize` hợp lý.